

Проект 5. REST API

В этом проекте ты поймёшь, как работает REST API и научишься отправлять запросы к реальному API портала «Работа России», получая актуальные данные о вакансиях напрямую от источника.

Содержание

- [Глава 1](#)
 - [Что такое REST API и как работает](#)
 - [Как читать JSON](#)
 - [Что смотреть в документации](#)
- [Глава 2](#)
 - [Практические задания](#)
 - [Задание 1. Исследование API и структуры ответа](#)
 - [Задание 2. Смотрим количество вакансий по запросу](#)
 - [Задание 3. Загрузка вакансий в таблицу](#)
 - [Задание 4. Проверка данных и визуализация по регионам](#)

Глава 1

Что такое REST API и как работает

Для начала работы в папке `src` скачай ноутбук `project_9.ipynb` — это твоя рабочая тетрадь на проект, для удобства в ней можно работать и запускать код.

REST API — это договорённость как обмениваться данными через интернет клиенту и серверу.

Клиент — это любая программа, которая инициирует взаимодействие. Она начинает «разговор», отправляя запрос на определённый адрес (URL).

Примеры клиентов:

- Веб-браузер, когда ты открываешь страницу сайта (он запрашивает HTML, CSS, данные).
- Мобильное приложение на телефоне (запрашивает ленту новостей, профиль пользователя).
- Скрипт на Python, который ты пишешь для сбора данных (запрашивает список вакансий).
- Postman или другая программа для тестирования API (ты вручную формируешь запрос).
- Другой сервер (микросервис), который запрашивает данные у твоего сервиса.

Сервер — это программа, которая постоянно работает и ждёт входящие запросы. Когда запрос приходит, сервер его обрабатывает и возвращает результат.

Примеры серверов:

- Веб-сервис (API), который отдаёт данные о вакансиях (например, `api.trudvsem.ru`).
- Бэкенд приложения, который управляет пользователями и заказами.
- Облачная функция (Cloud Function), которая запускается по запросу.
- Шлюз (API Gateway), который принимает запросы и перенаправляет их нужным сервисам.

Провайдер API принимает ключевые решения:

1. Какие сущности можно запрашивать:

- /users (пользователи),
- /vacancies (вакансии),
- /orders (заказы)
- и другие

2. Для каждого ресурса определяют доступные действия (методы HTTP):

- GET /users — получить список пользователей
- POST /vacancies — создать новую вакансию
- PUT — обновление существующего ресурса
- DELETE /users/{id} — удалить пользователя

Каждый метод — это чёткая команда. Сервер точно знает, что от него хотят, и отвечает предсказуемо.

3. Они описывают, что можно передавать в запросе для уточнения данных:

- номер страницы
- количество записей на странице
- поисковый запрос
- фильтр по региону

Ответ приходит в JSON-файл с определённой структурой.

Чтобы клиенты узнали о правилах, провайдер публикует документацию. Она может быть в разных форматах:

1. Текстовая документация на сайте, веб-страница с описанием параметров и примеров.
2. OpenAPI / Swagger — спецификация в формате YAML или JSON.
3. Готовая коллекция запросов, которую можно импортировать в Postman и сразу начать тестировать API без написания кода.
4. Готовый код (Python, JavaScript, Java), который скрывает детали HTTP-запросов.

Как читать JSON

Сервер и клиент могут быть написаны на разных языках, им нужен универсальный язык для обмена данными. Таким языком стал JSON (JavaScript Object Notation). Именно на этом языке сервер отвечает клиенту. JSON строится на двух главных структурах: объектах и массивах.

Базовые элементы JSON:

Объект — это набор пар «ключ: значение». Ключ — это название поля. Что может быть ключом:

```
{  
  "id": 123,  
  "job_name": "Аналитик данных",  
  "is_remote": true,  
}
```

```
"salary_min": 80000
}
```

В примере выше:

- "id", "job_name" — это ключи (названия полей)
- 123, "Аналитик данных" — это значения (то, что хранится в поле)
- Ключ и значение разделены двоеточием :
- Пары разделены запятыми

Что может быть значением:

- Строка: "Москва" (в двойных кавычках)
- Число: 80000, 3.14 (без кавычек)
- Логическое: true / false (без кавычек)
- null — значит «пусто» или «неизвестно»
- Другой объект: "salary": { "min": 80000 }
- Массив.

Массив — это упорядоченный список значений. Пригодится, когда нужно передать несколько однотипных объектов — например, список требований или список навыков.

Например:

```
[
  "Аналитик данных",
  "Python-разработчик",
  "Менеджер проекта"
]
```

Массив может содержать: Простые значения: ["Python", "SQL", "Tableau"] Объекты: [{ "id": 1 }, { "id": 2 }] Даже другие массивы: [[1, 2], [3, 4]]

```
# Пример чтения JSON
data = {
  "items": [
    {
      "name": "Аналитик данных",
      "employer": {
        "name": "Компания А"
      },
      "area": {
        "name": "Москва"
      }
    }
  ]
}
```

```
items = data.get("items", [])
first = items[0]

print("Название вакансии:", first.get("name"))
print("Работодатель:", first.get("employer", {}).get("name"))
print("Город:", first.get("area", {}).get("name"))
```

Что смотреть в документации

1. Находим Endpoint

Endpoint — это адрес API, к которому мы обращаемся. Выглядит примерно так:
`https://api.сервис.com/версия/ресурс`

Важно запомнить:

- Адрес часто содержит версию API (/v1/, /v2/).
- Ресурс обычно во множественном числе (/users, а не /user)
- Конкретный объект указывается через ID: /users/123

2. Method

Method — это действие, которое ты хочешь выполнить с ресурсом. Они всегда стандартные:

Метод	Когда использовать	Пример
GET	Получить данные (чтение)	Запросить список товаров
POST	Создать новый ресурс	Добавить нового пользователя
PUT	Полностью обновить ресурс	Заменить данные профиля
PATCH	Частично обновить ресурс	Изменить только email
DELETE	Удалить ресурс	Удалить запись

Но для каждого endpoint может быть ограниченное количество методов. Узнать какие конкретно методы доступны можно в документации API при наличии.

3. Изучаем Parameters

Parameters — это данные, которые ты передаёшь в запросе, чтобы уточнить, что именно тебе нужно.

Тип параметра	Где передаётся	Когда используется
Query	В URL после ?	Для фильтрации, поиска, пагинации (обычно в GET)
Path	Внутри самого URL	Для указания конкретного ресурса: /users/{id}
Body	В теле запроса	Для отправки данных при создании/обновлении (POST, PUT)

Тип параметра	Где передаётся	Когда используется
Header	В заголовках запроса	Для авторизации, указания формата и т.д.

4. Понимаем Response

Response — это то, что сервер вернёт в ответ на твой запрос.

Обрати внимание на эти моменты:

- Коды статусов. Смотри, какие коды может вернуть Это поможет тебе понять, успешен ли запрос и как реагировать на ошибки:

200 — запрос успешен 400 — ошибка в параметрах запроса 404 — ресурс не найден 429 — превышен лимит запросов 500 — ошибка на сервере и другие

A. Структура JSON. Изучи, как устроен ответ. Это нужно, чтобы знать, какие ключи будут в ответе и как к ним обращаться:

- Какие ключи есть на верхнем уровне?
- Есть ли вложенные объекты или массивы?
- Как называется список результатов: items, results, data?

B. Типы полей. Проверяй, что хранится в каждом поле. Это поможет понимать, что ожидать и как работать с данными в коде:

- Строка: "name": "Анна" — текст в кавычках
- Число: "price": 1200 — без кавычек, можно использовать в вычислениях
- Объект: "location": { "city": "Москва" } — вложенная структура, нужно «проваливаться» дальше
- Массив: "tags": ["python", "api"] — список, обращаемся по индексу [0]
- null: "middle_name": null — поле пустое, нужно проверять перед использованием

C. Пример ответа. Скопируй пример из документации и «пройди» по нему вручную — это нужно для тестов и отладки:

- Открой пример в текстовом редакторе
- Выпиши путь к нужным полям
- Проверь, какие поля могут отсутствовать (помечены как nullable или optional)
- Попробуй написать код извлечения данных на основе этого примера

```
import requests # подключаем библиотеку requests для отправки HTTP-запросов

# указываем адрес API, который возвращает данные о вакансиях
url = "https://opendata.trudvsem.ru/api/v1/vacancies"

# задаём параметры запроса
params = {
    "page": 1, # берём первую страницу результатов
```

```
"per_page": 3,      # просим вернуть 3 вакансии
"text": "кассир"    # ищем вакансии по слову "кассир"
}

# отправляем GET-запрос к API
response = requests.get(url, params=params)

# выводим статус ответа сервера
print("Статус ответа:", response.status_code)

# переводим JSON-ответ в Python-словарь
data = response.json()

# достаём список вакансий из ответа
vacancies = data.get("results", {}).get("vacancies", [])

# считаем, сколько вакансий пришло в ответе
print("Количество вакансий в ответе:", len(vacancies))

# выводим названия найденных вакансий
for item in vacancies:
    print(item.get("vacancy", {}).get("job-name"))
```

Глава 2

Практические задания

В этих практических заданиях ты будешь работать с API портала [«Работа России»](#)

Задание 1. Исследование API и структуры ответа

- Импортируй библиотеки: requests, pandas, matplotlib
- Сформируй запрос:
 - Адрес эндпоинта вакансий
 - Параметры: page=1, per_page=2, text="python"
- Отправь GET-запрос с таймаутом ~30 сек
- Проверь статус: выведи status_code, убедись, что он 200
- Распарси ответ: преобразуй JSON в структуру Python
- Изучи структуру:
 - Выведи ключи верхнего уровня ответа
 - Найди блок results, выведи его ключи
 - Извлеки список вакансий, проверь, что он не пустой
- Разбери первую запись:
 - Выведи ключи первого элемента списка
 - Найди внутри объект vacancy, выведи его ключи
- Зафиксируй: выпиши названия полей, которые понадобятся для анализа

Задание 2. Смотрим количество вакансий по запросу

- Отправь GET-запрос с параметрами:

- `page=1`, `per_page=1` (достаточно для получения числа)
 - `text="аналитик"`
- Распарси ответ и выведи ключи верхнего уровня
- Найди общее число вакансий в поле `meta.total`
- Выведи результат: "Найдено вакансий: X"
- Повтори запрос для слова "python" и сравни числа

Задание 3. Загрузка вакансий в таблицу

- Отправь GET-запрос с параметрами:
 - `page=1`, `per_page=10` (загружаем 10 вакансий)
 - `text="аналитик"`
- Распарси ответ и извлеки список вакансий
- Для каждой из 10 вакансий извлеки 5 ключевых полей:
 - `job-name` (название должности)
 - `company.name` (компания)
 - `region.name` (регион)
 - `salary_min` (минимальная зарплата)
 - `creation-date` (дата публикации)
- Создай список словарей с этими полями: `job_name`, `company_name`, `region`, `salary_min`, `published_at`
- Преобразуй список в DataFrame через `pd.DataFrame()`
- Выведи:
 - Размер таблицы через `.shape`
 - Первые 5 строк через `.head()`

Задание 4. Проверка данных и визуализация по регионам

- Проверь структуру данных:
 - Выведи список колонок таблицы
 - Посчитай количество пропущенных значений в каждой колонке
 - Отсортируй результат по убыванию (чтобы видеть самые «проблемные» поля)
- Подготовь данные для графика:
 - Возьми колонку с регионом (`region`)
 - Замени пропущенные значения на "Не указано"
 - Посчитай количество вакансий в каждом регионе
 - Оставь только топ-10 регионов по количеству
- Построй круговую диаграмму:
 - Создай фигуру размером 8×8 дюймов
 - Построй pie-график с отображением процентов (`autopct="%1.1f%%"`)
 - Добавь заголовок «Распределение вакансий по регионам»
 - Убери подпись оси `ylabel`, чтобы график выглядел аккуратнее
 - Отобрази график